

Señales en POSIX

Malak Sanchez

Monitorias Sistemas Operativos

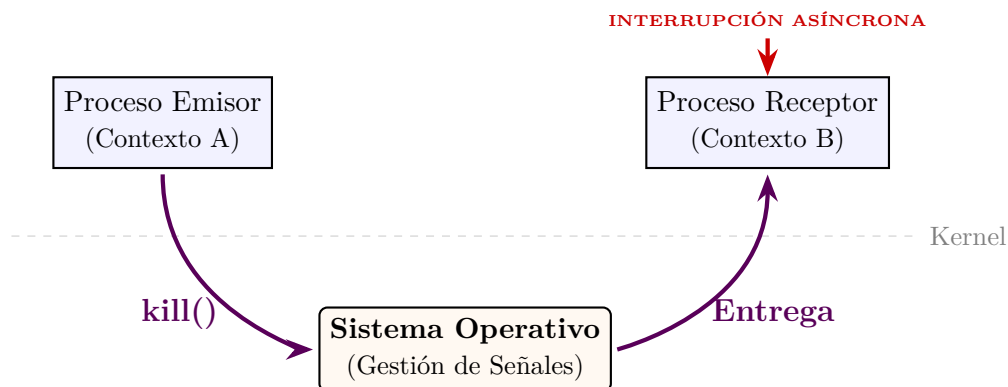
22 de septiembre de 2026

1. Motivación

Las señales son notificaciones asíncronas enviadas a un proceso para informarle de un evento específico. Son el mecanismo de IPC más ligero y rápido, permitiendo interrumpir la ejecución normal del programa para atender situaciones críticas (como un error de memoria) o eventos externos (como una interrupción del usuario). Se rigen bajo el estándar POSIX.1.

2. Idea Fundamental

Una señal es un mensaje corto enviado por el kernel o por otro proceso. Cuando un proceso recibe una señal, el flujo de ejecución normal se interrumpe para ejecutar una acción específica.



Cuando un proceso recibe una señal, puede:

- Ignorarla (excepto SIGKILL y SIGSTOP).
- Ejecutar el manejador por defecto (ej: terminar el proceso).
- Ejecutar una función personalizada (`signal_handler`).

3. Sintaxis Básica: Capturando Señales

Aunque existen dos formas de manejar señales, la función canónica es `signal()`.

```
#include <signal.h>
typedef void (*handler_t)(int);
sighandler_t signal(int signo, handler_t handler);
void signal_handler(int sig); //función a ejecutar al recibir la
    señal
```

Descripción de los parámetros

- **signo:** El número o constante de la señal (ej: `SIGINT`).
- **handler:** Un puntero a la función que se ejecutará (**handler**). También puede ser `SIG_IGN` (ignorar) o `SIG_DFL` (acción por defecto).

4. Señales Comunes y Acciones

Cada señal tiene una acción por defecto definida por el kernel:

Señal	ID	Acción Defecto	Evento
<code>SIGINT</code>	2	Terminar (Interrupt)	Interrupción de teclado (Ctrl+C).
<code>SIGKILL</code>	9	Terminar (Kill)	Ejecución forzosa (Ineludible).
<code>SIGSEGV</code>	11	Terminar (Core Dump)	Fallo de segmentación (Memoria).
<code>SIGTERM</code>	15	Terminar	Solicitud de terminación amable.
<code>SIGCHLD</code>	17	Ignorar	Un proceso hijo ha terminado.
<code>SIGUSR1</code>	10	Terminar	Señal de usuario 1 (Uso libre).
<code>SIGUSR2</code>	12	Terminar	Señal de usuario 2 (Uso libre).

Importancia de `SIGUSR1` y `SIGUSR2` Estas señales son fundamentales en el curso porque son las únicas reservadas explícitamente para que el programador defina su comportamiento. A diferencia de `SIGINT` o `SIGSEGV`, el kernel no les asigna un evento de hardware o de sistema específico; por lo tanto, son las herramientas perfectas para simular protocolos de comunicación personalizados entre procesos.

5. Ejemplo Mínimo: Capturando SIGINT (Ctrl+C)

```
1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 void signal_handler(int sig) {
6     printf("\nReceived signal %d. I won't die easily!\n", sig);
7 }
8
9 int main(){
10     signal(SIGINT, signal_handler);
11     printf("Process waiting for signals (PID: %d). Press Ctrl+C\n", getpid());
12     while(1){
13         pause(); // Wait for a signal to arrive
14     }
15     return 0;
16 }
```

6. Pro-Tip: Funciones Seguras (Reentrancia)

Un error común es usar `printf()` dentro de un manejador de señales. Técnicamente, `printf()` no es **async-signal-safe**; si el programa es interrumpido mientras `printf()` está ejecutándose, llamar a otro `printf()` puede causar un *deadlock* o corrupción de buffers. En código de producción, se prefiere usar `write()`.

7. Enviando Señales: `kill()`

Para enviar una señal desde código C se utiliza la función `kill()`. Esta función permite enviar cualquier señal a otro proceso.

```
#include <sys/types.h>
#include <signal.h>

int kill(pid_t pid, int sig);
```

Descripción de los parámetros

- **pid**: El identificador del proceso al que se envía la señal.
- **sig**: El número o constante de la señal (ej: `SIGUSR1`).

8. Ejercicios

1. Modifique el ejemplo para que el programa termine solo después de recibir `SIGINT` cuatro veces.

2. Cree un programa padre que cree un hijo. El hijo debe instalar un manejador para `SIGUSR2`, informar que está listo y esperar; el padre debe enviarle la señal después de 2 segundos con `kill()`.
3. Intente ignorar la señal `SIGSTOP`. ¿Es posible? Justifique su respuesta después de probarlo.